

Mushroom Yield Forecasting System

Polyhouse Sensor Data · Machine Learning Pipeline · Streamlit Deployment

Author: Lekshmi Biju

GitHub: github.com/LekshmiBiju/mushroom-yield-project

Live App: mushroom-yield-project-7nxbbbjsufc83coh3l6vz3.streamlit.app



Introduction

WHAT THIS PROJECT DOES

- Predicts daily mushroom yield (kg) from polyhouse sensor readings
- Compares Linear Regression vs Random Forest models
- Deploys a live Streamlit app for real-time yield forecasting
- Implements prediction logging and monitoring pipeline
- Built across a 22-day structured ML bootcamp

TECH STACK

Python 3.10

scikit-learn

Random Forest

Streamlit

pandas

MinMaxScaler

joblib

PROJECT SCOPE

- Domain: Agritech / Polyhouse Farming
- Data: Daily sensor readings (2024)
- Target: yield_kg (continuous)
- Split: 80% train / 20% test (time-ordered)
- Evaluation: MAE, RMSE, R^2
- Deployment: Streamlit Community Cloud

Problem Statement

Research Question

Can we accurately predict daily mushroom yield (kg)
from polyhouse environmental sensor readings?

Environmental Sensitivity

- Mushroom yield is highly sensitive
- to temperature, humidity, and CO₂
- Small changes affect production significantly

Manual Estimation

- Farmers rely on visual inspection
- No data-driven prediction tool existed
- Harvest planning is reactive, not proactive

AgriTech Opportunity

- Sensor data is already collected
- ML can convert raw readings to forecasts
- Enables informed daily harvest decisions

Objective: Build a regression model predicting yield_kg · Deploy with Streamlit · Log every prediction

Dataset Overview

Total Rows

365

daily readings

Features

3

sensor inputs

Target

yield_kg

kg per day

Date Range

365

days (2024)

Missing

0

values

DATASET COLUMNS			SUMMARY STATISTICS (RAW DATA)		
Column	Description	Type			
timestamp	Date of reading	datetime			
temperature_c	Polyhouse temp (°C)	float64	temperature_c	18.15 – 26.37 °C	
humidity_pct	Relative humidity (%)	float64	humidity_pct	78.10 – 94.80 %	
co2_ppm	CO ₂ concentration (ppm)	float64	co2_ppm	608 – 1154 ppm	
yield_kg	Daily mushroom yield (kg)	float64 ← TARGET	yield_kg	15.31 – 18.85 kg	

Data Cleaning

01

Ingestion & Parsing

- Read polyhouse_sensors.csv with parse_dates
- Cast all sensor columns to float64
- Saved to data/interim/01_loaded.parquet

02

Validation Rules

- Humidity kept between 0–100%
- CO₂ must be > 0 ppm
- Temperature range: 10–35 °C
- yield_kg must be non-null

03

Missing & Duplicates

- 0 missing values detected
- Forward fill applied (limit=2) on sensor cols
- Duplicate timestamps removed (keep=last)
- Result: 365 → 365 rows (0 dropped)

04

Output & Logging

- Cleaned data → data/interim/02_cleaned.parquet
- cleaning_log.md records all steps
- data_quality.md generated with full stats
- Missing value report saved as CSV

Pipeline: src/ingest.py → src/clean.py → data/interim/ · All steps reproducible and logged

EDA Findings

KEY INSIGHTS

- Humidity shows strongest positive relationship with yield ($r = 0.24$)
- Temperature has moderate positive correlation with yield ($r = 0.52$)
- CO₂ shows slight negative relationship with yield ($r = -0.26$)
- Temperature stable: 24.5–26 °C in EDA subset (limited variance)
- No significant outliers observed in any feature
- All 365 rows passed humidity and CO₂ validation checks
- Dataset suitable for regression modeling

CORRELATION WITH yield_kg



4 VISUALIZATIONS CREATED

- Correlation Heatmap
- Humidity vs Yield Scatter
- Temperature vs Yield Scatter
- CO₂ vs Yield Scatter

DATA QUALITY SUMMARY

- 0 missing values
- 365 / 365 rows valid
- 100% humidity rule pass
- 100% CO₂ rule pass
- No duplicates found

Feature Engineering

FEATURES USED IN FINAL MODEL

Feature	Type	Description
temperature_c	Raw sensor reading	Temperature inside polyhouse (°C)
humidity_pct	Raw sensor reading	Relative humidity percentage (%)
co2_ppm	Raw sensor reading	Carbon dioxide concentration (ppm)
temp_humid_interaction	Engineered	temperature_c × humidity_pct / 100 (interaction term)

SCALING: MinMaxScaler

- Applied MinMaxScaler to all 4 features
- Scaler fitted on training set only (prevent leakage)
- Same scaler transforms test set
- Output range: [0, 1] for all features
- Saved: models/minmax_scaler.joblib
- Also saved: models/minmax_scaler_train.joblib

TRAIN / TEST SPLIT

- Data sorted by timestamp (time-series order)
- 80% train: 292 rows (Jan–Oct 2024)
- 20% test: 73 rows (Oct–Dec 2024)
- No random shuffle — preserves temporal order
- Avoids data leakage from future into past
- Stored: data/processed/X_train.npy, X_test.npy

Models Compared

Two models were trained and evaluated: Linear Regression (baseline) and Random Forest (tuned with GridSearchCV).
TimeSeriesSplit cross-validation was used throughout to respect temporal ordering of the data.

Linear Regression

Baseline

Test MAE: 0.46 kg
Test RMSE: 0.58 kg
Test R²: 0.34
Train MAE: 0.42 kg
Train R²: 0.43

- Simple, interpretable baseline
- Assumes linear relationship
- Features: temp, humidity, CO₂
- MinMaxScaler applied
- src/linear_regression.py

Random Forest (Default)

n_estimators=100

Test MAE: 0.45 kg
Test RMSE: 0.57 kg
Test R²: 0.33
CV MAE: 0.474 ±0.057

- Ensemble of 100 decision trees
- Handles non-linear patterns
- random_state=42, n_jobs=-1
- Same features as Linear
- src/random_forest.py

Random Forest (Tuned) ★

GridSearchCV

Test MAE: 0.44 kg
Test RMSE: 0.56 kg
Test R²: 0.37
CV MAE: 0.469
CHAMPION MODEL

- n_estimators=50, max_depth=8
- min_samples_leaf=3
- TimeSeriesSplit CV scoring
- Lowest MAE & highest R²
- Saved: champion.joblib

Tuned RF uses: n_estimators=50, max_depth=8, min_samples_leaf=3 · No overfitting: CV MAE ≈ Test MAE

Model Evaluation



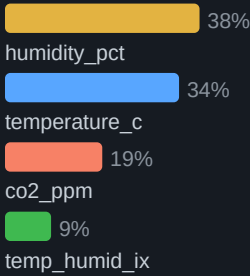
VALIDATION STRATEGY

- TimeSeriesSplit (5 folds) cross-validation
- Preserves temporal order — no future leakage
- 80/20 chronological train/test split
- GridSearchCV with MAE scoring
- CV MAE close to test MAE → good generalisation
- Overfitting analysis: no severe overfitting found

GRIDSEARCH PARAMETERS

n_estimators	[50, 100, 200]
max_depth	[None, 8, 16]
min_samples_leaf	[1, 3, 5]
Best: n_estimators	50
Best: max_depth	8
Best: min_samples_leaf	3

FEATURE IMPORTANCE (RF)



Results

Champion MAE

0.44 kg

Tuned RF test set

Champion RMSE

0.56 kg

Tuned RF test set

Champion R²

0.37

Tuned RF test set

Improvement

−4.3%

MAE vs baseline LR

MODEL COMPARISON TABLE

Model	MAE	RMSE	R ²
Linear Regression	0.46	0.58	0.34
Random Forest (Default)	0.45	0.57	0.33
Random Forest (Tuned) ★	0.44	0.56	0.37

RESIDUAL ANALYSIS FINDINGS

- Residuals centered around zero — no systematic bias
- No major curvature pattern in residual vs predicted
- Residual spread reasonably consistent across range
- residual_vs_predicted.png — homoscedastic pattern
- residual_vs_humidity.png — no humidity-driven bias
- pred_vs_actual.png — predictions track actual yield
- Linear Regression acceptable as baseline per diagnostics

Streamlit App Demo



Monitoring & Logging

PREDICTION LOGGING — src/logger.py

timestamp_utc	ISO format UTC timestamp of each prediction
temp_c	Temperature input from user slider
humidity_pct	Humidity input from user slider
co2_ppm	CO ₂ input from user slider
predicted_kg	Model output rounded to 3 decimal places

Sample log entry: 2024-06-22T14:30:00+00:00, 22.0, 88.0, 920, 16.983

MONITORING STRATEGY

- Monitor prediction outputs weekly
- Check for invalid/out-of-range sensor inputs
- Verify prediction consistency over time
- Log stored to: logs/predictions.csv

RETRAINING TRIGGERS

- > 100 new records collected
- Prediction accuracy decreases
- Environmental conditions change significantly
- Monthly review schedule reached

DRIFT THRESHOLDS

- Temperature: 10 – 35 °C valid range
- Humidity: 70 – 100 % valid range
- CO₂: 400 – 2000 ppm valid range
- App warns user if inputs are out of range

Future Improvements

1. Real IoT Sensors

Connect live polyhouse sensors via MQTT/API. Eliminate manual CSV collection and enable truly real-time prediction.

2. Larger Dataset

365 rows is limited. Collect multi-year or multi-farm data to improve model generalization and R^2 score.

3. Automated Retraining

Deploy monthly retraining pipeline triggered by new data accumulation (> 100 new records) with performance check.

4. User Auth Dashboard

Add login-based multi-user dashboard so multiple farm operators can track their own yield histories securely.

5. Mobile App

Convert Streamlit app to React Native or Flutter mobile app for in-field access without laptop dependency.

6. Additional Features

Add substrate type, spawn age, watering frequency, and light hours as additional predictive features.

Lessons Learned

TOP 3 SKILLS GAINED

1 End-to-End ML Pipeline

Built the full data science workflow from raw CSV → cleaning → EDA → feature engineering → modelling → deployment → monitoring in a structured 22-day bootcamp.

2 Time-Series Awareness

Learned why random shuffling is wrong for time-ordered data. Used chronological split and TimeSeriesSplit CV to avoid leakage from future into training.

3 Streamlit Deployment

Turned a local model into a live, public web app. Learned how to cache models, display predictions, add warnings, build sensitivity charts, and log every prediction.

REFLECTION

2 AREAS TO GROW

1 Larger & Richer Dataset

365 rows with only 3 features limited the model R^2 to 0.37. Next step is multi-year sensor data and additional environmental features (substrate, light, pH) to improve predictive power.

2 Deep Learning for Time-Series

LSTMs or Transformer models could capture sequential patterns in yield data that tree-based models miss. Would require more data and GPU compute, but is the natural next ML step.

Real-world ML is 80% data work and 20% modelling. The biggest lesson was that a small, clean dataset with a simple model still produces a useful, deployable product — and that logging, validation, and reproducibility matter as much as R^2 .

Conclusion

Mushroom Yield Forecasting System — successfully built and deployed

- ✓ 365-row polyhouse sensor dataset cleaned, analysed, and modelled
- ✓ 2 models compared: Linear Regression vs Tuned Random Forest
- ✓ Tuned Random Forest selected as champion (MAE 0.44 kg, R^2 0.37)
- ✓ Feature engineering: temp × humidity interaction + MinMaxScaler
- ✓ Live Streamlit app deployed on Streamlit Community Cloud
- ✓ Prediction logging to CSV with timestamp, inputs, and output
- ✓ Full 22-day bootcamp capstone — Task 10 completed

GitHub: github.com/LekshmiBiju/mushroom-yield-project

Live App: mushroom-yield-project-7nxbbbjsufc83coh3l6vz3.streamlit.app

Author: Lekshmi Biju · AI Data Analyst (Beginner) Bootcamp 2026